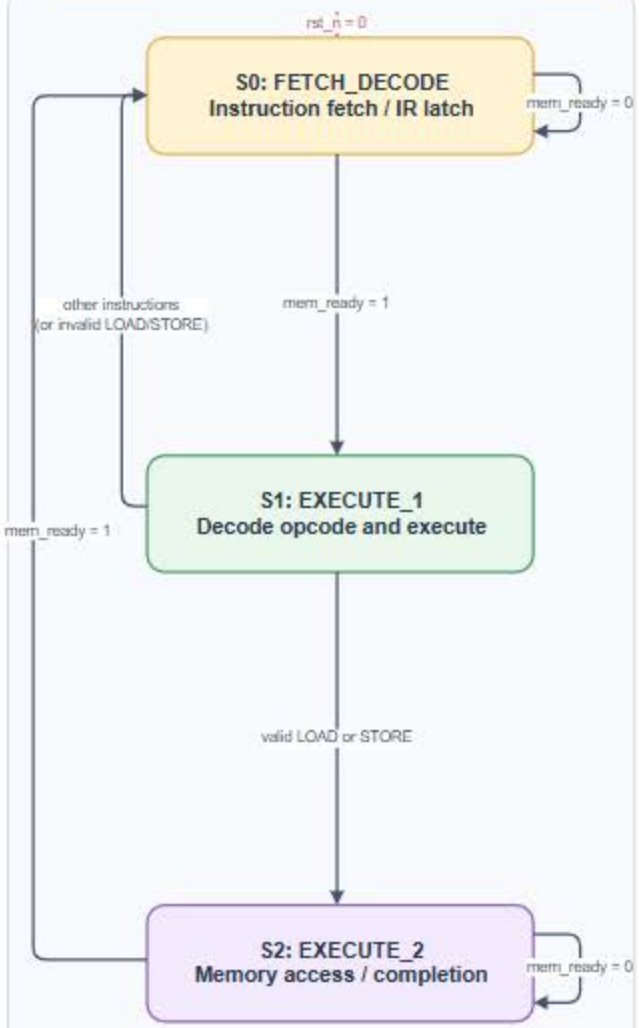
RISC-V CPU CONTROLLER | FSM STATE DIAGRAM

STATE TRANSITIONS



TRANSITION RULES:

S0: 0 -> S0;
1 -> S1 (mem_ready)

S1: valid LOAD/STORE -> S2;
else -> S0

S2: 0 -> S2;
1 -> S0 (mem_ready)

Reset -> S0

CONTROL ENCODING LEGEND

PCSel[1:0]	00: PC+4	01: PC+IMM	10: JALR	
ALUBSel[1:0]	00: RS2	01 IMM:	10: unused PSM	
WBSel[2:0]	000 ALU	001: Load	010: PC+4	
	011 U_IMM	100: PC_IMM		
MemWriteMode	00 NONE	01: SB	10: SH	11: SW
RFWrite_n	0: register write			

DEFAULT CONTROL OUTPUTS

PCWrite	0	ALUASel	0 (rs1)
PCSel[1:0]	00 (PC+4)	ALUBSel[1:0]	00 (RS2)
IRWrite	0	ALUOp[3:0]	0000 (ADD)
RFWrite_n	1 (no write)	WBSel[2:0]	000 (ALU)
EAWrite	0	MemRead	0
AddrSel	0	MemWriteMode[1:0]	00 (NONE)

CONTROLLER INPUTS

clk / rst_n	1 bit each
mem_ready	1 bit
opcode / funct3 / funct7	7 / 3 / 7 bits
flags = {overflow,negative,zero}	3 bits; zero_flag=flags[0]

CONTROLLER OUTPUTS (12)

PCWrite / IRWrite / EAWrite	1 bit each
RFWrite_n / AddrSel / ALUASel	1 bit each
PCSel / ALUBSel / MemWriteMode	2 bits each
ALUOp / WBSel / MemRead	4 / 3 / 1 bits

S0 · FETCH_DECODE | Memory handshake

AddrSel	0 (PC)
MemRead	!mem_ready
IRWrite	mem_ready

mem_ready = 0 → stay in FETCH_DECODE (request memory read)
mem_ready = 1 → latch IR at clock edge, transition to EXECUTE_1

S1 · EXECUTE_1 | Opcode-dependent outputs and ALU operation

R-type (OP) (opcode=0110011)

ALUASel	0 (rs1)
ALUBSel	00 (rs2)
ALUOp	decode funct3 / funct7
WBSel	000 (ALU)
RFWrite_n	0 (write)
PCWrite / PCSel	1 / 00

Next: S0

I-type Arithmetic (OP-IMM) (opcode=0010011)

ALUASel	0 (rs1)
ALUBSel	01 (IMM)
ALUOp	decode funct3 / funct7
WBSel	000 (ALU)
RFWrite_n	0 (write)
PCWrite / PCSel	1 / 00

Next: S0

LUI (opcode=0110111)

WBSel	011 (U_IMM)
RFWrite_n	0 (write)
PCWrite	1
PCSel	00 (PC+4)

Next: S0

AUIPC (opcode=0010111)

WBSel	100 (PC_IMM)
RFWrite_n	0 (write)
PCWrite	1
PCSel	00 (PC+4)

Next: S0

LOAD (opcode=0000011)

valid_load	funct3=000/001/010/100/101
ALUASel	0 (rs1)
ALUBSel	01 (IMM)
ALUOp	0000 (ADD)
EAWrite	1 (save address)
PCWrite	0

valid -> S2; invalid: PCWrite=1, PCSel=00 -> S0

STORE (opcode=0100011)

valid_store	funct3=000/001/010
ALUASel	0 (rs1)
ALUBSel	01 (IMM)
ALUOp	0000 (ADD)
EAWrite	1 (save address)
PCWrite	0

valid -> S2; invalid: PCWrite=1, PCSel=00 -> S0

JAL (opcode=1101111)

WBSel	010 (PC+4)
RFWrite_n	0 (write)
PCWrite	1
PCSel	01 (PC+IMM)

Next: S0

JALR (opcode=1100111)

Condition	funct3 = 000
ALUASel / ALUBSel	0 / 01 (IMM)
ALUOp	0000 (ADD)
WBSel / RFWrite_n	010 (PC+4) / 0
PCWrite / PCSel	1 / 10 (ALU→JALR)

funct3=000: PCWrite=1, PCSel=00; Next: S0

BRANCH (opcode=1100011)

Common: ALUASel / ALUBSel	0 / 00 (rs1, rs2)
PCWrite	1
PCSel default	00 (PC+4)

Next: S0; PCSel=01 only if branch condition true

Branch condition table | zero_flag = flags[0]

000	BEQ	0001 SUB	PCSel=01 if zero=1
001	BNE	0001 SUB	PCSel=01 if zero=0
100	BLT	0100 SLT	PCSel=01 if zero=0
101	BGE	0100 SLT	PCSel=01 if zero=1
110	BLTU	0110 SLTU	PCSel=01 if zero=0
111	BGEU	0110 SLTU	PCSel=01 if zero=1

ALUOp decoding for R-type / I-type

000	ADD or SUB	funct7=0100000 -> SUB; else ADD
001	SLL	0010
010	SLT	0100
011	SLTU	0110
100	XOR	1000
101	SRL / SRA	funct7=0100000 -> SRA; else SRL
110	OR	1100
111	AND	1110

Unsupported opcode in EXECUTE_1: RFWrite_n=1, PCWrite=1, PCSel=00, Next=S0
Unsupported LOAD/STORE funct3: PCWrite=1, PCSel=00, Next=S0

S2 · EXECUTE_2 | Load / Store memory access (AddrSel = 1 for whole state)

LOAD wait: mem_ready=0 (opcode=0000011)

AddrSel	1 (EA)
WBSel	001 (Memory)
MemRead	1
RFWrite_n	1 (no write)
PCWrite	0

Next: S2

LOAD complete: mem_ready=1 (opcode=0000011)

AddrSel	1 (EA)
WBSel	001 (Memory)
MemRead	0
RFWrite_n	0 (write)
PCWrite / PCSel	1 / 00(PC+4)

Next: S0

STORE wait: mem_ready=0 (opcode=0100011)

AddrSel	1 (EA)
MemWriteMode[1:0]	01: SB (funct3=000)
	10: SH (funct3=001)
	11: SW (funct3=010)
PCWrite	0

Next: S2

STORE complete: mem_ready=1 (opcode=0100011)

AddrSel	1 (EA)
MemWriteMode	00 (NONE)
PCWrite / PCSel	1 / 00

Next: S0

EXECUTE_2 unexpected opcode: PCWrite=1, PCSel=00; next_state = S0 if mem_ready=1, else S2.